# Scaling PyTorch on Intel XPU

**Field notes from AI for Science on Aurora**

Sam Foreman · Argonne National Laboratory

Panagiotis Kourdis · Intel

Tanima Dey · Intel

*PyTorch Conference · October 21, 2026 — San Jose*

Interactive version:
live diagrams + links

samf.sh/talks/2026/10/21

Argonne NATIONAL LABORATORY

intel

PyTorch CONFERENCE NORTH AMERICA 2026

# Aurora at ALCF

| | |
|---|---|
| **ALCF** | DOE leadership computing — runs `Aurora` + `Polaris` for open science |
| **Aurora** | `10,624` nodes · `127,488` XPUs |
| **This talk** | Running `PyTorch` on `Aurora`: backend fixes, training, and recovery |




**Fig:** *Aurora at the ALCF. Photo: Argonne / ALCF.*

Here, **XPU** = one GPU tile exposed as a `torch.xpu` device in `FLAT` mode. **12 XPUs per node.**



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Talk outline

| Part | Question | Implementation |
|---|---|---|
| **A · Needed** | What does it take to build a GPU that runs PyTorch?<br>*What about 100,000 of them, together?* | ~~IPEX~~ (legacy) · `torch-xpu-ops` ·<br>`XCCL / torch-ccl`<br>`oneAPI` · `oneCCL` · `SYCL` |
| **B · Accomplished** | How do you run AI for Science campaigns at scale?<br>*On a brand-new, rapidly evolving software stack?* | `ezpz` · `Megatron-DeepSpeed` · `torchtitan/ezpz`<br>`DTensor` · `FSDP2` |
| **C · Remains** | What keeps a 14-day job alive at 60k → 120k XPUs? | *Nobody knows!*<br>Best bets: `ezpz checkpoint-restart` · watchdog ·<br>node exclusion · scheduler resubmit |



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Scientific AI on Aurora

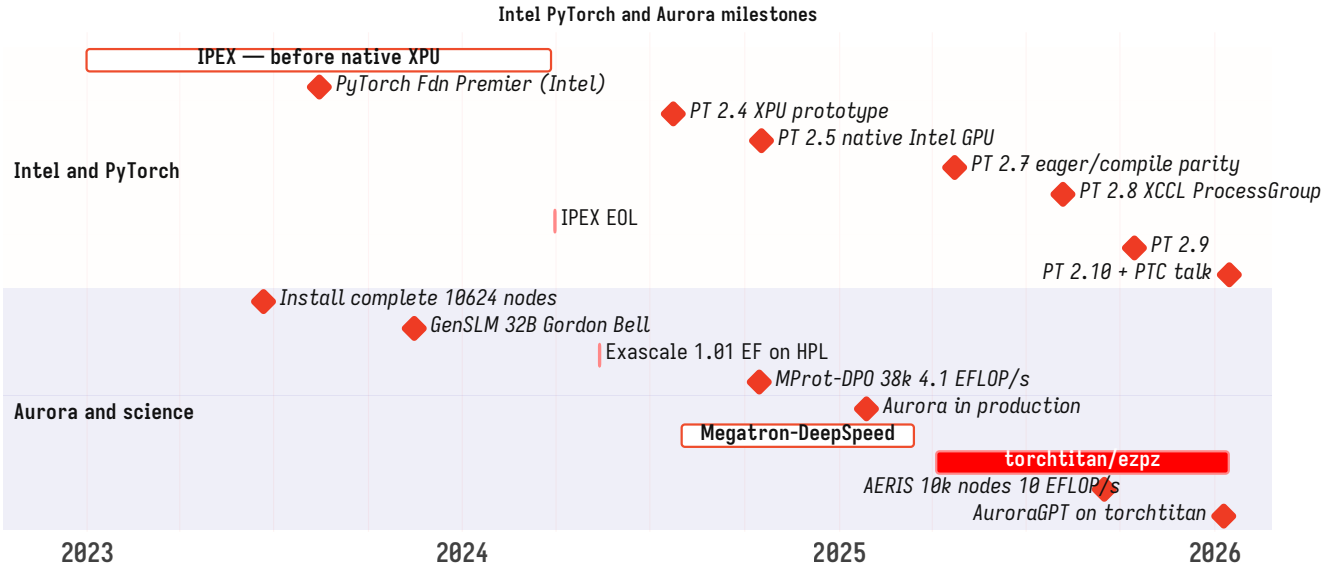
| Workload | Scale / result |
|---|---|
| GenSLM — viral genomes | Polaris + Cerebras · Gordon Bell |
| MProt-DPO — protein design | 38,400 ranks · 4.11 EFLOP/s at 3,200 nodes |
| AERIS — weather diffusion | 10,080 nodes · 10.21 EFLOP/s · 90-day stable rollouts |
| AuroraGPT — foundation LLM | 7.77T-token `MDS` reference → torchtitan |

`ezpz` : same training program on `Intel` , `NVIDIA` , `AMD` .[1]



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## PyTorch XPU and Aurora timeline



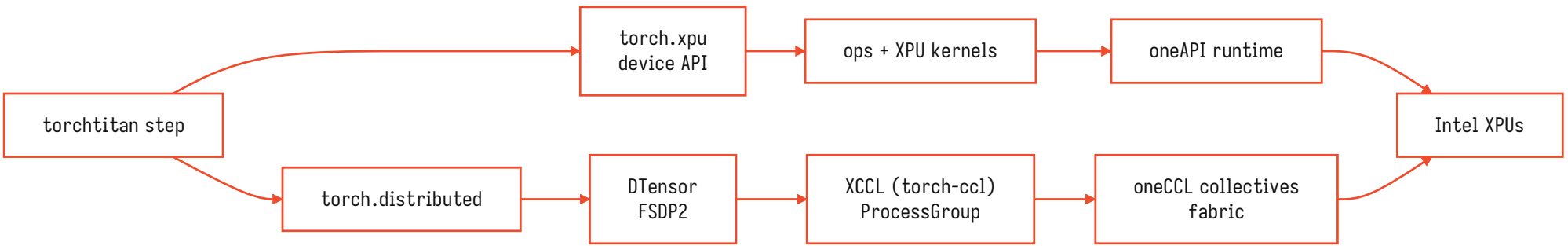

- **Intel and PyTorch:** IPEX → PT 2.4–2.10+XPU (4 releases)
- **Aurora and science:** 10k nodes → 120k-XPU workloads

From ezpz: distributed PyTorch across any hardware plus verified Aurora dates.



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## PyTorch XPU backend components




Test the device API, kernels, and collectives together. `IPEX` shims → native `torch.xpu` + `torch-ccl` / `XCCL` + `oneCCL`



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## XPU integration issues

| Symptom | Root cause | What we run now |
|---|---|---|
| `torch 2.13`: `expected DTensor, got plain tensor` on FSDP wrap | Missing pytorch#181519 | Fixed in `2.14.0+xpu` |
| `DeviceMesh` → `does not support splitting` | `ProcessGroupXCCL` inherits `supportsSplitting() == false` (see pytorch#186548) | Local shim · pytorch#186548 |
| `train_timeout_seconds` is a no-op | `_set_pg_timeout` has no `xpu` branch — `XCCL` stores the timeout, never arms it | Watchdog heartbeat (Act C) + two-PR upstream |

The `CUDA` path worked. The `XPU` path inherited defaults that needed fixing.



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## From Megatron-DeepSpeed to torchtitan

| Megatron-DeepSpeed fork [2] | torchtitan + DTensor + FSDP2 |
|---|---|
| 2B trained to 7.77T tokens (Aug talk) | Same run continues on torchtitan — 4.67T at 256 → 512 nodes |
| Parallelism baked into the fork | DTensor + FSDP2 as composable primitives |
| Every PT release → rebase + patch audit | XPU integration in separate layers · torchtitan/ezpz |

The fork trained the 2B model to **7.77T tokens**. Each PyTorch upgrade required another rebase and patch audit.



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## One training program across vendors

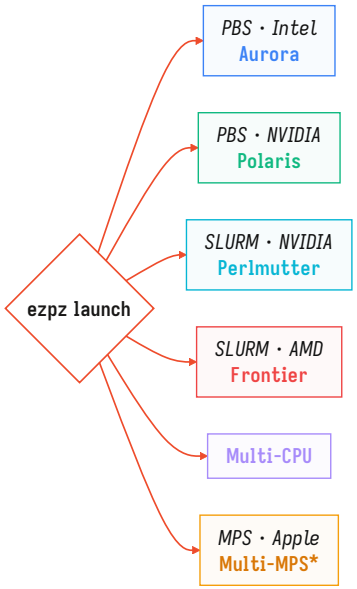
**Hello world:**

```python
# example.py
import torch
import ezpz
rank = ezpz.setup_torch()
world_size = ezpz.get_world_size()
dev = ezpz.get_torch_device()      # {xpu, cuda, cpu, ...}
# FSDP2 by default — accepts DeviceMesh
model = ezpz.wrap_model(torch.nn.Linear(128, 10).to(dev))
print(f"Hello from {rank}/{world_size}!")
```

```bash
$ ezpz launch python3 example.py
Hello from rank 0/24!
Hello from rank 1/24!
# ... etc
```

▶ Other examples

Same command on every machine — `ezpz` picks the backend.




* `mccl` — *mps-ddp/mccl:* `MPS` *on Apple silicon —* `ezpz launch` *even on a laptop.*



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Benchmark the training step

1. Fix the build, model, batch size, precision, and node count.
2. Run the same training step on each backend.
3. Measure step time, tokens/s, collective time, and memory.
4. Increase node count. Rerun after kernel or collective changes.

> An op test can pass while the distributed training step hangs.

Check loss and throughput at each node count. Test timeout and recovery behavior separately.[3]



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Changes in the `torchtitan` training stack

| Layer | Before (MDS fork + `ezpz`) | After (`torchtitan/ezpz`) |
|---|---|---|
| Parallelism | Custom tensor-parallel + pipeline in fork | `FSDP2` + `DTensor` sharding |
| Collectives | DeepSpeed's `NCCL` assumptions | `XCCL` (`torch-ccl`) → `oneCCL` |
| Kernels | Fork-local fused kernels | `XPU` kernels via `torch.xpu` — `layernorm+residual` moved; `attn` / `MLP` / `optimizer` remain[4] |
| Launch | Already `ezpz` — scheduler → nodes → `MPI` | Same `ezpz`[5] — `launch` + `checkpoint-restart` |
| Checkpoint | `DeepSpeed` engine save | `DCP` (`torch.distributed.checkpoint`) |

`ezpz` stayed in place. Parallelism, collectives, and checkpoints moved out of the fork.



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Testing at every layer

| Layer | Gate | Link |
|---|---|---|
| Numerics | 50-step loss — `±0.10` on final + last-10 mean | Production loss figure |
| Collectives | XCCL ladder `48 → 768` ranks, value-checked | `ezpz/test_dist.py` |
| Parallelism | TP1 / TP2 / MoE on Aurora; parity on Sunspot | `experiments/ezpz` |
| Portability | One gate on Aurora + Sunspot + Polaris + Perlmutter | `ezpz` |
| Orchestration | Failover harness — 9 fixtures | Failover writeup |

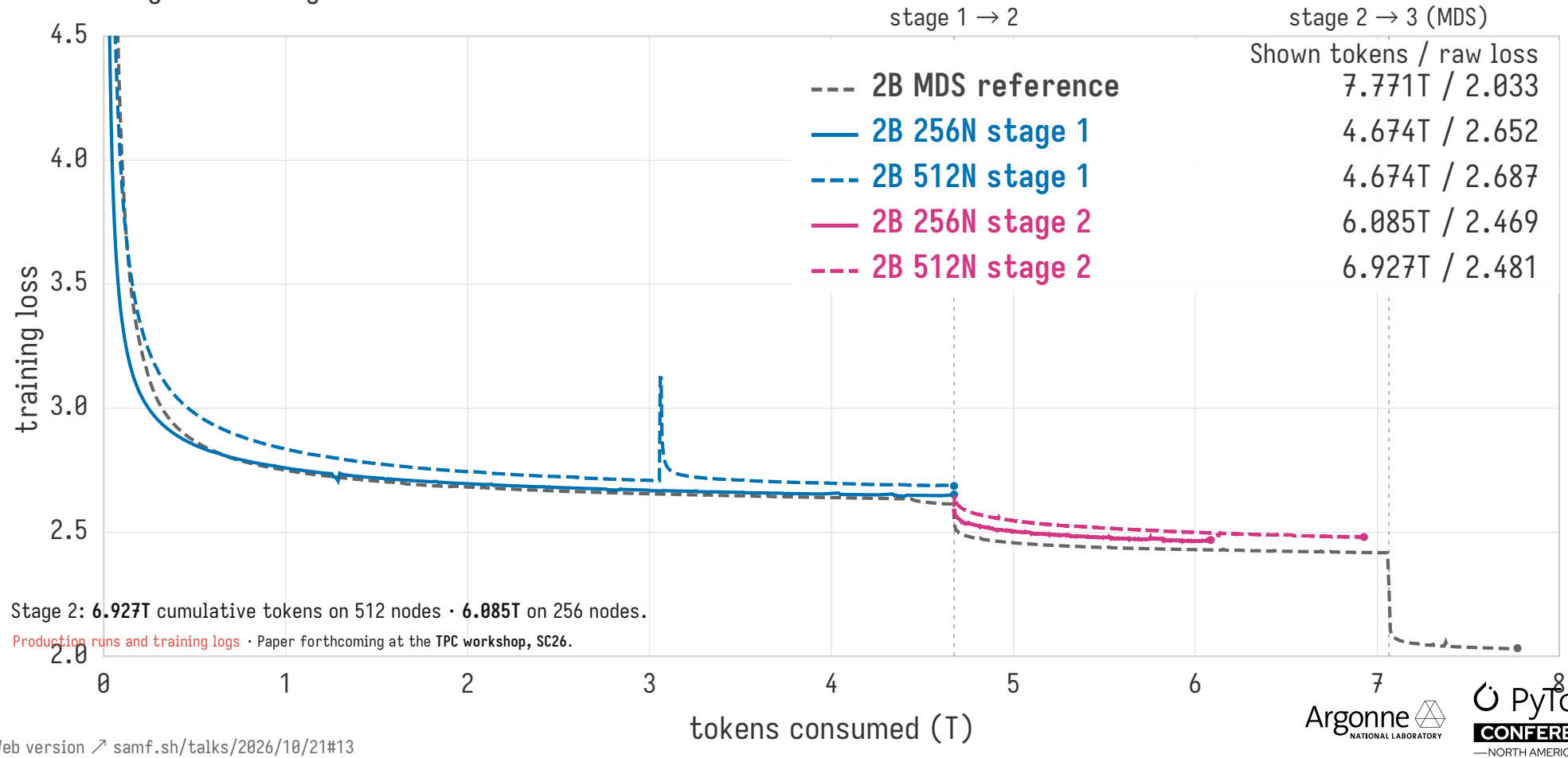
The streaming loader contributes `~0.05` loss variation from shuffle order. The 50-step check uses a `±0.10` tolerance.



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

# AuroraGPT 2B: production training progress

Stage 1 and stage 2 · cumulative tokens




Stage 2: **6.927T** cumulative tokens on 512 nodes · **6.085T** on 256 nodes.

Production runs and training logs · Paper forthcoming at the **TPC workshop, SC26.**

Argonne NATIONAL LABORATORY

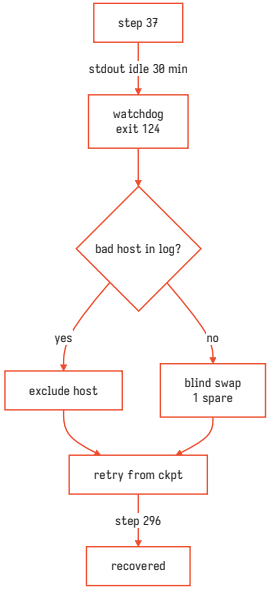
PyTorch CONFERENCE NORTH AMERICA 2026

Web version ↗ samf.sh/talks/2026/10/21#13

Solid: 256 nodes · dashed: 512 nodes. Stage 2 includes stage-1 tokens; training mixes differ.

## Detecting and recovering from stalled jobs

**Job** `8505298` — step 37, then silence. No traceback. Watchdog at 30 min (`exit 124`), wrapper swapped a node, relaunched. Attempt 2 reached step 296.




`_set_pg_timeout` has no `xpu` branch — timeout is stored, never enforced.[6]



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Failure modes at ~120k XPUs

| Mode | Signal | What helps | What doesn't |
|---|---|---|---|
| **Silent hang** | Stdout stalls, no error | Watchdog on log progress | `_set_pg_timeout` (`xpu` never arms) |
| **Lost collective** | Rank times out in `XCCL` | Node exclude + retry | Retrying on the same host |
| **Crash / OOM** | Traceback + exit code | Process restart from `ckpt` | Scheduler resubmit alone |
| **Stale FS / DCP** | Checkpoint read stalls | Async save + local staging | Sync save on shared `FS` at 10k ranks |

Test node exclusion, checkpoint reload, and scheduler resubmission before a long run.[7]

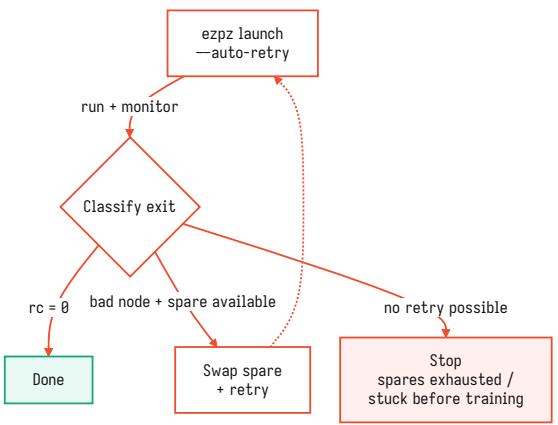


Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Process, node, and allocation restarts

| Layer | Scope | Trigger | Cost of being wrong |
|---|---|---|---|
| Process | 1 rank | Crash / OOM | Tokens since last ckpt |
| Node | 12 XPUs | Hang on one host | 1 node's work + re-shard |
| Allocation | 10k nodes | Bad FS / scheduler | Whole-job walltime |

`ezpz` handles node and allocation restarts: `checkpoint-restart` + node exclude + scheduler resubmit. `DCP` writes the checkpoints.
Test each restart path before committing a full allocation.




Exit trailer + crash patterns + idle watchdog (exit 124) determine the next action.



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Remaining XPU gaps

| Layer | Gap | What's needed |
|---|---|---|
| Kernels | Fused attn + MLP + optimizer across XPU | Composable kernels compatible with `torch.compile` |
| Collectives | Tuning, diagnostics, **real timeouts** | `XCCL` timeout enforcement + `DTensor` split fix upstream |
| Checkpoints | Async `DCP` at 10k ranks without `FS` collapse | Local staging → async flush |
| Operations | Detect bad nodes during the run | Heartbeat at the `ProcessGroup` level, not stdout `grep` |

Run the training step, save a checkpoint, interrupt the job, and resume it.



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Bringing up the next accelerator

1. Test compute and collectives together.
2. Use the same training program on each machine.
3. Compare loss against the reference run.
4. Measure full training steps at scale.
5. Interrupt, restart, and check the resumed loss.



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Links and further reading

| Stack | Link |
|---|---|
| `ezpz` | Launch · Checkpoint · All docs |
| `torchtitan/ezpz` | Experiments · Failover writeup |
| PyTorch XPU | `torch.xpu` · `DTensor` · `FSDP2` · DCP |
| Distributed | `torch-ccl` / XCCL · `oneCCL` · torch.distributed |
| System | Aurora · ALCF support — Aurora · ALCF news — 120k-tile scale |
| Science | AuroraGPT companion · AERIS · MProt-DPO · GenSLM |

Further reading: `ezpz` portability · Launching 50k processes · Data and training at ALCF



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Acknowledgements

This work used resources of the **Argonne Leadership Computing Facility**, a DOE Office of Science user facility at Argonne National Laboratory, supported under contract **DE-AC02-06CH11357**.
**Aurora** was built in partnership with **HPE** and **Intel** — Intel Max 1550 XPU ( `PVC` ), `oneAPI` / `oneCCL` / `XCCL` , and the upstream `torch.xpu` work tracked in Act A. Thanks to the ALCF operations and support teams, the `PyTorch` and `torchtitan` maintainers, and the science teams behind `GenSLM` , `MProt-DPO` , `AERIS` , and `AuroraGPT` .

| | |
|---|---|
| **Facility** | ALCF · Aurora · Polaris |
| **Partners** | HPE · Intel (Max 1550 / `oneAPI` / `oneCCL` ) · PyTorch Foundation |
| **Teams** | `GenSLM` · `MProt-DPO` · `AERIS` · `AuroraGPT` — see Links for papers |



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Aurora allocation programs

**Anyone** (!) can apply for time on any of the `ALCF` systems, including our AI Testbed!

| Program | Best for | Cadence |
|---|---|---|
| **INCITE** | Large, peer-reviewed science campaigns | Annual |
| **ALCC** | DOE-mission-aligned projects | Annual |
| **Director's Discretionary (DD)** | Porting, benchmarking, getting started — **start here** | Rolling, weeks |

For a `DD` proposal, find me after the talk: `foremans@anl.gov` / `@saforem2`.

ALCF allocations overview · DD program details · ALCF support center



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Next at ALCF: Solstice and Equinox

Announced with NVIDIA and Oracle:

| System | Planned accelerators |
|---|---|
| **Equinox** | **10,000** NVIDIA Blackwell GPUs |
| **Solstice** | **100,000** NVIDIA Blackwell GPUs |

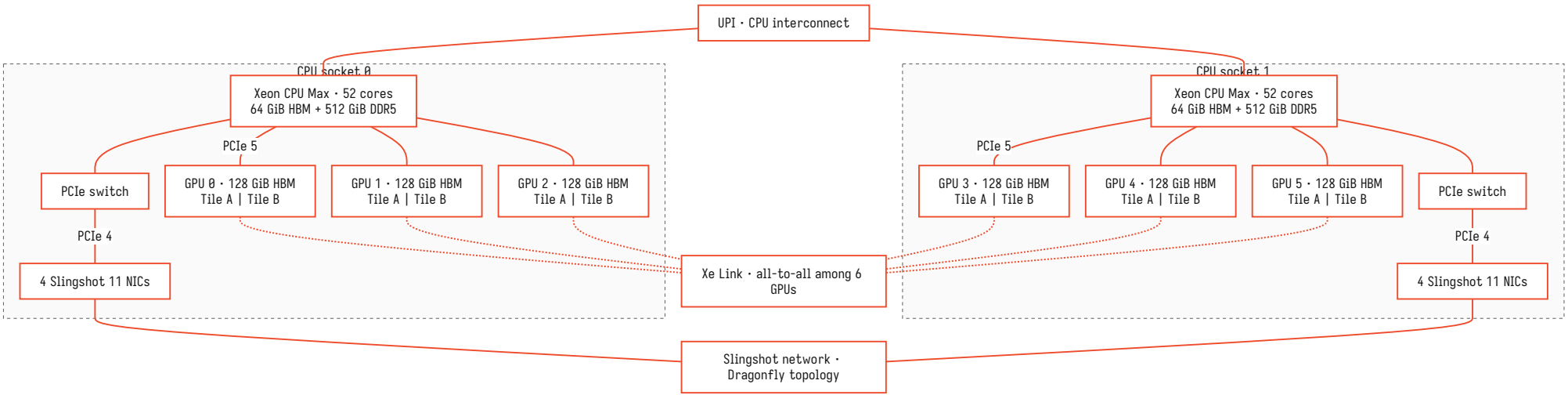
The hardware, fabrics, launchers, networks, and file systems keep changing.
The same training code still needs to launch, communicate, checkpoint, and recover at scale.

NVIDIA / Oracle / DOE announcement · October 2025



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

## Backup: inside an Aurora node




**6 GPUs × 2 tiles = 12 `torch.xpu` devices** in `FLAT` mode. GPU Max 1550 (`PVC`), 768 GiB GPU HBM per node.

Logical schematic, not a wiring diagram. Xe Link connections grouped. ALCF node topology · Machine overview · PyTorch tile configuration



Argonne NATIONAL LABORATORY
PyTorch CONFERENCE —NORTH AMERICA 2026—

# Session feedback

Scan to share feedback with us and the organizers.

Scaling PyTorch on Intel XPU: Field Notes from AI for Science on Aurora

Web version ↗ samf.sh/talks/2026/10/21#24

Argonne NATIONAL LABORATORY

PyTorch CONFERENCE —NORTH AMERICA 2026—